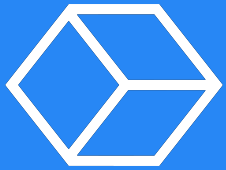


Introduction to neural networks

Laboratory of robotics and control systems
ITT PROJECT MANAGEMENT SERVICES L.L.C

Plan

1. Notible architectures

2. Improving mobility

3. Saving resources

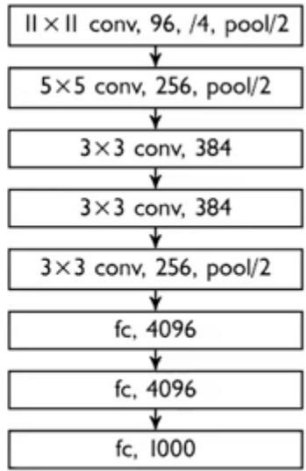
AlexNet

Layers

- Conv = Conv2d + ReLU + Pooling
- Dropout (p=0.5) in FC part
- Heavy FC layers after CNN
- Large convs at the first layers, big strides (reducing feature maps size)
- Large fully connected layers

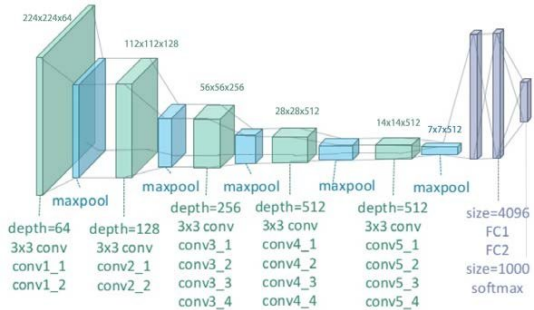
Features

- Learning with GPU (5-6 days on two NVIDIA GTX 580 3GB GPUs)
- Data augmentations

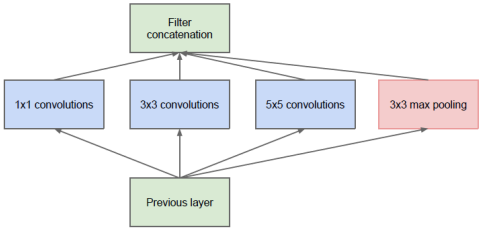


VGG

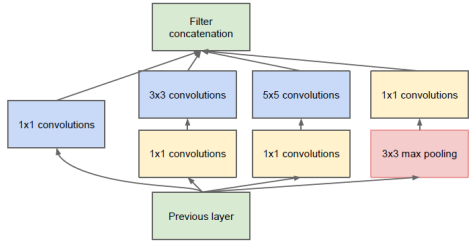
- Similar to AlexNet
- Only 3x3 convs, stride 1
- Increase depth
- Increase filters by 2 times after each pooling
- Adding layers while learning



Inception



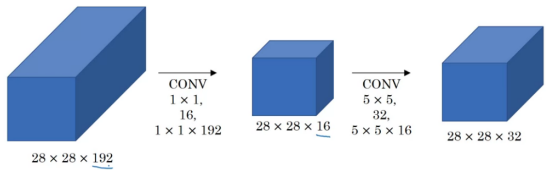
(a) Inception module, naïve version



(b) Inception module with dimension reductions

1x1 convolution

- Expand or reduce tensor depth

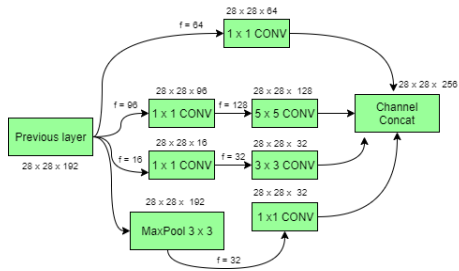


Inception

- Using 1x1 convs to reduce num of params
- Increase depth (22 layers)
- Faced with fading gradients
- Multiple outputs to solve gradient troubles

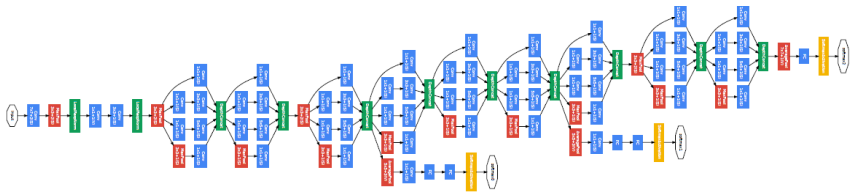
$$L = \alpha L_1 + \beta L_2 + \gamma L_3$$

type	patch size/ stride	params	ops
convolution	7×7/2	2.7K	34M
max pool	3×3/2		
convolution	3×3/1	112K	360M
max pool	3×3/2		
inception (3a)		159K	128M
inception (3b)		380K	304M
max pool	3×3/2		
inception (4a)		364K	73M
inception (4b)		437K	88M
inception (4c)		463K	100M
inception (4d)		580K	119M
inception (4e)		840K	170M
max pool	3×3/2		
inception (5a)		1072K	54M
inception (5b)		1388K	71M
avg pool	7×7/1		
dropout (40%)			
linear		1000K	1M
softmax			



Inception

- **Inception-v2 (2015)**

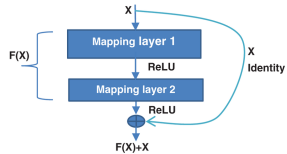


- **Inception-v2 (2015)** adding BatchNorm and shortcut convolutions
- **Inception-v3 (2015)** adding dilated convolutions
- **Inception-v4 (2016)** adding residual connections (spoiler)

Residual block

- Learning the adding to input function
- $H(X) = F(X) + X$
- $\frac{\partial H(X)}{\partial X} = \frac{\partial F(X)}{\partial X} + \frac{\partial X}{\partial X}$
- solve problem of vanishing gradients

VGG-19, "VGG-34" - plain
VGG-19 and "VGG-34" with skip cons - ResNets



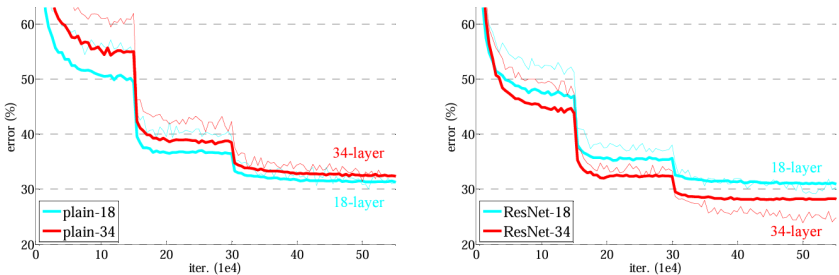
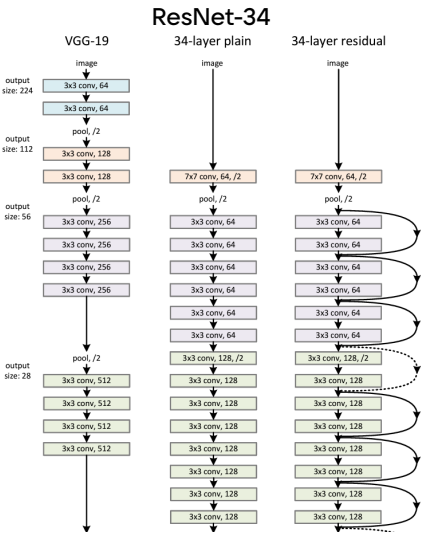


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

ResNet



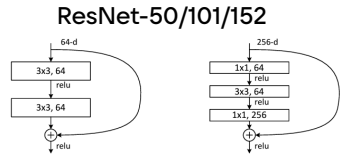


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a "bottleneck" building block for ResNet-50/101/152.

- Add expansion and reduction tensor depth by 1x1 convolutions
- Depth is increased
- There are fewer parameters than in VGG

To sum up

ILSVRC (ImageNet Large Scale Visual Recognition Challenge) results

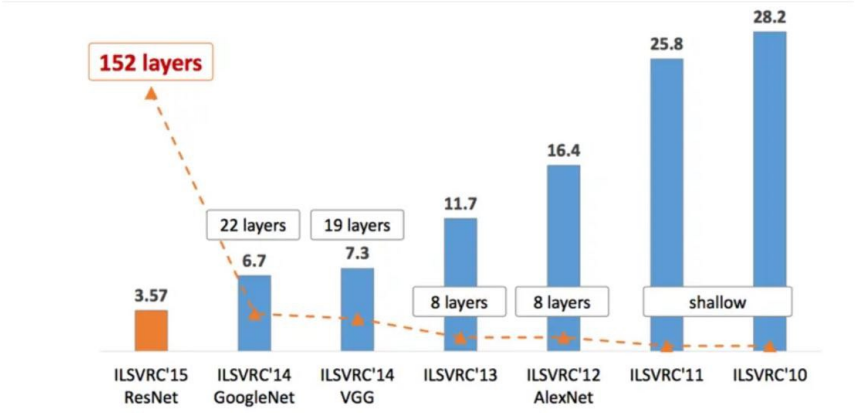
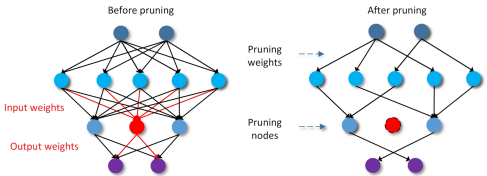


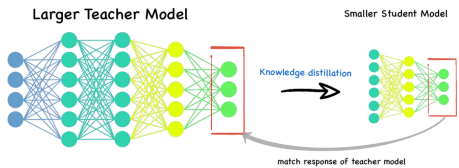
Table of Contents

Improve efficiency

Pruning



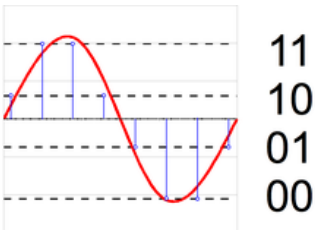
Distillation



Model A
Accuracy: 98%
Runtime: 1.5 sec
Size: 200 mb

Model B
Accuracy: 96%
Runtime: 0.2 sec
Size: 20 mb .

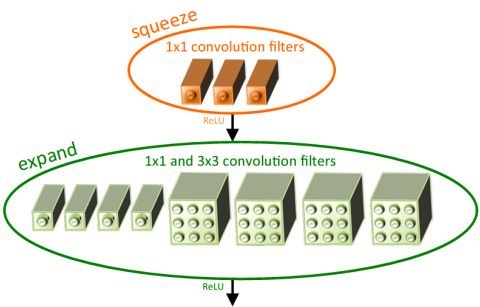
Quantizing

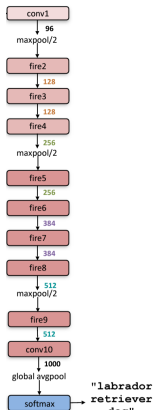


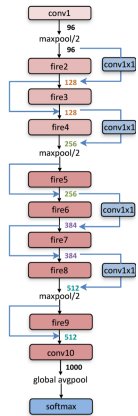
- To come up with new architectures

SqueezeNet

- squeeze and expand - **Fire module**
- Targets are Alexnet's output probabilities vector
- No FC layers







SqueezeNet

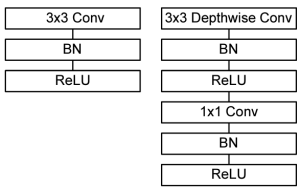
Table 2: Comparing SqueezeNet to model compression approaches. By *model size*, we mean the number of bytes required to store all of the parameters in the trained model.

CNN architecture	Compression Approach	Data Type	Original → Compressed Model Size	Reduction in Model Size vs. AlexNet	Top-1 ImageNet Accuracy	Top-5 ImageNet Accuracy
AlexNet	None (baseline)	32 bit	240MB	1x	57.2%	80.3%
AlexNet	SVD (Denton et al., 2014)	32 bit	240MB → 48MB	5x	56.0%	79.4%
AlexNet	Network Pruning (Han et al., 2015b)	32 bit	240MB → 27MB	9x	57.2%	80.3%
AlexNet	Deep Compression (Han et al., 2015a)	5-8 bit	240MB → 6.9MB	35x	57.2%	80.3%
SqueezeNet (ours)	None	32 bit	4.8MB	**50x**	57.5%	80.3%
SqueezeNet (ours)	Deep Compression	8 bit	4.8MB → 0.66MB	**363x**	57.5%	80.3%
SqueezeNet (ours)	Deep Compression	6 bit	4.8MB → 0.47MB	**510x**	57.5%	80.3%

- Are small models amenable to compression?
- SOTA results from the model compression community are surpassed
- Inference time is decreased
- This compressed model was implemented on FPGA. (fully downloaded to FPGA memory)
- **Deep Compression:** compressing deep neural networks with pruning, trained quantization and Huffman coding

MobileNet

- **depthwise separable convolution:** depthwise convolution + pointwise convolution
- depthwise separable convolutions requires less params and repeat convolutions
- show the tradeoffs of reducing the network based on the two hyper-parameters: width multiplier and resolution multiplier



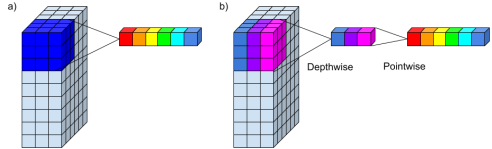


Table 6. MobileNet Width Multiplier

Width Multiplier	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
0.75 MobileNet-224	68.4%	325	2.6
0.5 MobileNet-224	63.7%	149	1.3
0.25 MobileNet-224	50.6%	41	0.5

Table 7. MobileNet Resolution

Resolution	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
1.0 MobileNet-192	69.1%	418	4.2
1.0 MobileNet-160	67.2%	290	4.2
1.0 MobileNet-128	64.4%	186	4.2

EfficientNet

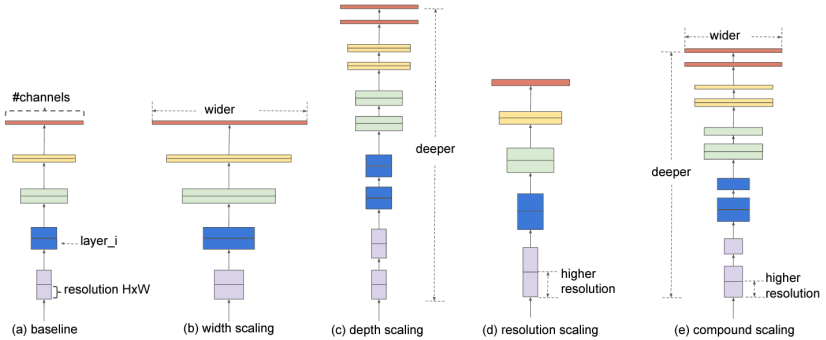


Figure 2. **Model Scaling.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

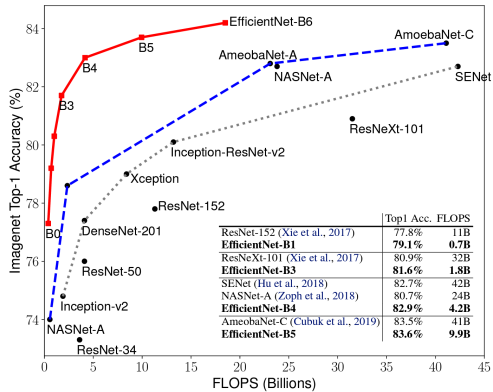
- **compound scaling method** is proposed

$$\text{depth: } d = \alpha^{\phi}, \quad \text{width: } w = \beta^{\phi}, \quad \text{resolution: } r = \gamma^{\phi}$$

$$\text{s.t. } \alpha * \beta^2 * \gamma^2 \approx 2, \quad \alpha \geq 1, \quad \beta \geq 1, \quad \gamma \geq 1$$

EfficientNet

- balancing network width, depth, and resolution
- easily scale up a baseline ConvNet to any target resource constraints, while maintaining model efficiency
- main building block is mobile inverted bottleneck MBConv (Sandleretal.,2018; Tanetal.,2019) with squeeze-and-excitation optimization



Sources

- Alexnet
- VGG
- Inception
- ResNet
- SqueezeNet: Alexnet-level accuracy with 50x fewer parameters and <0.5 Mb model size
- MobileNet
- EfficientNet
- review

Table of Contents

Transfer Learning, Fine-tuning

Idea:

- Use feature extractor (backbone) that was learnt on large datasets in your purposes.
- Initialize only top layers (head).

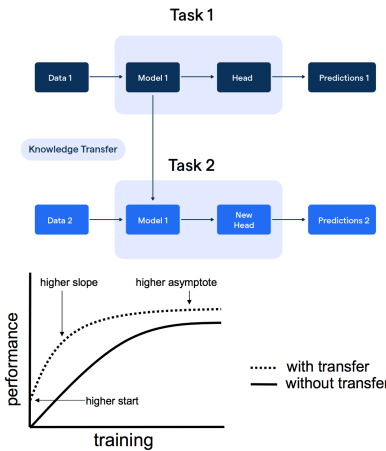
Pre-trained model: trained on large datasets model.

Variants:

- **Transfer Learning**
  - freezes all pre-trained layers
  - only top layers are trained
- **Fine-tuning**
  - both pre-trained layers and top layers are trained

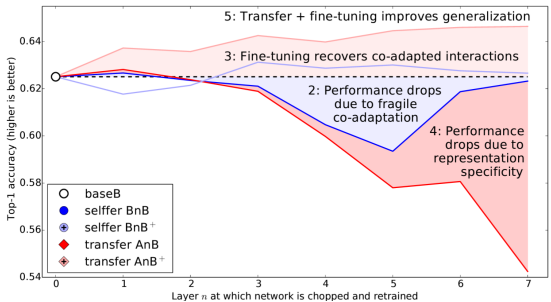
Main goal:

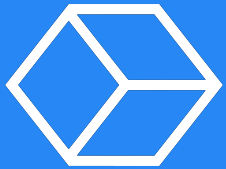
- better initial model
- better metric values
- less data is used
- faster training



Paper on fine-tune

- ImageNet is splited into 2 parts (first 500 classes, second 500 classes).
- Eight-layer convolutional networks
- Model A is trained on first 500 classes
- Model B is trained on second 500 classes
- A on second 500 classes with locking bottom layers(AnB)
- A on second 500 classes without locking bottom layers (AnB^+)
- First n layers are transfered. Others are initialized.
- Paper





Thank you for your time

Laboratory of robotics and control systems
ITT PROJECT MANAGEMENT SERVICES L.L.C